BMB201. NESNE YÖNELİMLİ PROGRAMLAMA

Ders 6: Nesne Yönelimli Programlama II

Polymorphism (Çok-şekillilik)

- NYP'de ise bir nesnenin kendi özellikleri dışına çıkıp başka bir nesne gibi davranmasına polymorphism denir. Gerçek dünyada Polymorphism örnekleri:
  - Örnek 1:
    - Öğretmenin öğrencilerine davranışı
    - Öğretmenin meslektaşlarına davranışı
  - Örnek 2:
    - Öğrencilerin evde davranışları
    - Öğrencilerin sınıf içindeki davranışları
  - Örnek 3: Bir cep telefonu tek isme sahip ama çok şekle sahip olması
    - Telefon
    - Camera
    - MP3 Çalar
    - Radio

Polymorphism Türleri

- Statik Polymorphism (Compile Time)
  - Function Overloading
    - Constructor Overloading
  - Operator Overloading
- Dinamik Polymorphism (Run Time)
  - Kalıtım (Inheritance)
    - Virtual
    - Abstract
    - Interface Polymorphism
    - Sealed

Statik Polymorphism (Compile Time)

- Bir nesne derleme zamanında function'lar arasındaki bağlantı mekanizmasını inşa eder. Bu yönteme erken bağlama (early binding) veya statik bağlama (static binding)'te denir.
  - Function overloading
  - Operator overloading

Function overloading (Fonksiyon Aşırı Yükleme)

- Aynı sınıf içinde aynı isme sahip birden fazla function tanımlayabilme / aşırı yükleme imkanı sağlar.
- Fakat, function'ların içerdiği veri tipleri veya function'ın parametre sayısı (argüman sayısı) farklı olmalıdır.
- Diğer taraftan aynı girdi özelliklerine sahip fakat farklı return tipine sahip bir function aşırı yüklenemez.

Function overloading

```
public class islem
{
    #region Parametre Sayısının Farklı Olması
    //parametre sayısı farklı
    public int topla()
    {
        return 0;
    }

    public int topla(int sayi)
    {
        return sayi;
    }

    public int topla(int sayi1, int sayi2)
    {
        return sayi1 + sayi2;
    }

    public int topla(int sayi1, int sayi2, int sayi3)
    {
        return sayi1 + sayi2 + sayi3;
    }
    #endregion

    Parametre Türlerinin Farklı Olması
}
```

- islem sınıfı aynı isme sahip (toplam), benzer parametre tiplerine sahip (integer) fakat parametre sayıları birbirinden farklıdır.
- islem sınıfında iki region'a ayrıldı. (Region konusunu hatırla)

Function overloading

```
public class islem
{
    Parametre Sayısının Farklı Olması

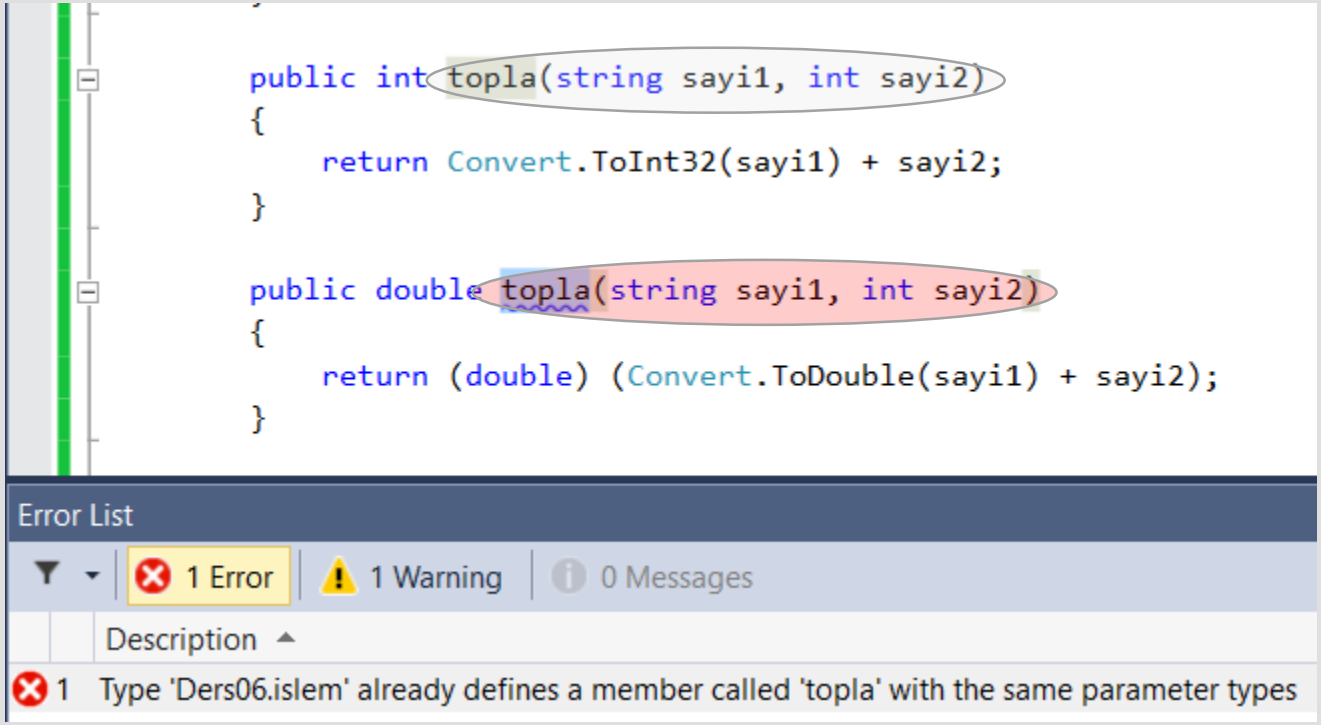
    #region Parametre Türlerinin Farklı Olması
    //parametre türü farklı
    public int topla(string x)
    {
        return Convert.ToInt32(x);
    }

    public int topla(string sayi1, int sayi2)
    {
        return Convert.ToInt32(sayi1) + sayi2;
    }

    public int topla(int[] ss)
    {
        int t = 0;
        for (int i = 0; i < ss.Length; i++)
            t += ss[i];
        return t;
    }
    #endregion
}
```

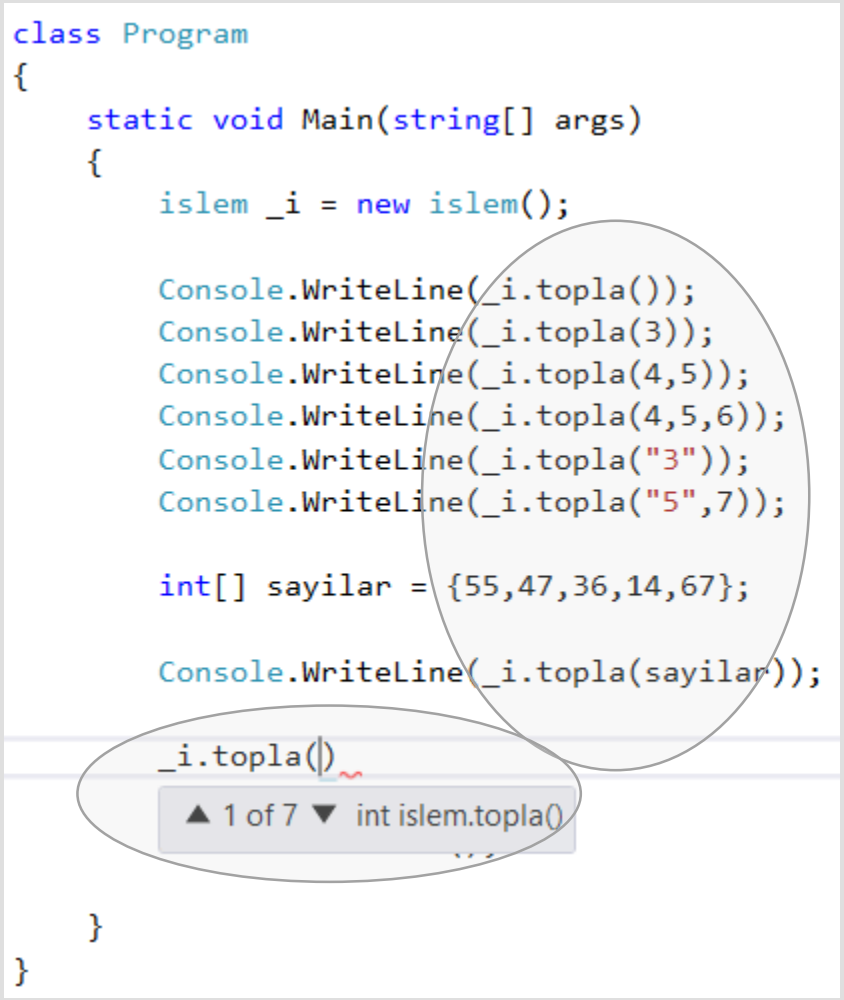
- Aynı değeri döndüren farklı türlere sahip topla function'ı

Function overloading



- Aynı parametre özelliklerine sahip ama farklı değer döndüren function tanımlanamaz.
- Hata mesajına dikkat!!!

Function overloading



- Bir sınıf yaratılıp o sınıf içinden aynı isme ait functionlara yandaki gibi erişilebilir.
- Nesne ismini yazıp Parantez açtığınızda 7 function'a dikkat.
- Console içindeki WriteLine function'ında benzer overloading durumu vardır.
- Statik veya harici sınıfa function overloading uygulanabilir.

Constructor Overloading

- Constructor'larda functionlara benzer şekilde overload edilebilir. Örneğin:

```
public class X
{
    private int m;

    public X()
    {
        m = 0;
    }

    public X(int k)
    {
        m = k;
    }

    public X(int k, int z)
    {
        m = k + z;
    }

    public int m_get()
    {
        return m;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        X _x01 = new X();

        Console.WriteLine(_x01.m_get());

        X _x02 = new X(6);

        Console.WriteLine(_x02.m_get());

        X _x03 = new X(6, 6);

        Console.WriteLine(_x03.m_get());

        Console.ReadLine();
    }
}
```

Operator Overloading

- Uygulama geliştirirken aslında operatörleri çok sık şekilde kullanmaktayız.
- Örneğin iki integer değişkenin arasına toplama (+) operatörü toplayıp başka bir integer değişken içine atarız. Sonuç iki değişkenin toplamıdır.
- Console.WriteLine içinde + operatörünü bir string bir de integer değer için kullandığımızda ise toplama işlemi olmaz. Toplama işlemi yerine integer'da string gibi davranır ve string birleştirme işlemi yapılmış olur. İki string içinde aynı durum geçerlidir. (C'de iki string + ile birleştirilebiliyor muydu? Bir fonksiyon ile yapılıyordu)
- Operatör Kullanma
  - .Net Hazır operatörlerini kullanabiliriz. (DateTime Örneği vereceğiz.)
  - Kendi sınıflarımıza kendimize özel operatörler düzenleyebiliriz.

Operator Overloading (DateTime Örneği)

```
class Program
{
    static void Main(string[] args)
    {
        DateTime _baslangic_dt = new DateTime();
        _baslangic_dt = DateTime.Now;

        for (int i = 0; i < 1000000000; i++)
        {

        }//zaman kaybı için

        DateTime _bitis_dt = new DateTime();
        _bitis_dt = DateTime.Now;

        Console.WriteLine("Başlangıç zamanı: " + _baslangic_dt.ToString());
        Console.WriteLine("Bitiş zamanı: " + _bitis_dt);

        TimeSpan _ts = _bitis_dt - _baslangic_dt;
        Console.WriteLine(_ts);
        Console.ReadKey();
    }
}
```

+ operatörü İki string birleştiriliyor.

+ operatörü bir string ve bir DateTime birleştiriliyor. Sonuç string.

- operatörü İki DateTime çıkarıyor ve TimeSpan diye bir sınıf içine sonucu atıyor. Normal çıkarmadan farklı.

- DateTime .Net'in System içindeki sınıflarından biridir. Yukarıdaki kod işlemin başlangıç ve bitişi bulup aradaki fark işlemini yapmaktadır.

Kişisel Operatör Tanımlama

- Karmaşık sayı iki bölümden oluşan ve her iki bölümü ayrı işlem gören bir matematiksel işlemdir. İki constructor'dan oluşan ve private x, y değerlerini bir constructor'lar üzerinden alan ve yaz adında bir fonksiyonla string tipinde karmaşık sayıyı döndüren bir sınıf tasarlandık.
- Operatörlerimiz region bu sınıf için tanımlanan operatörleri bulabilirsiniz.

```
public class karmasiksayi
{
    private int x;
    private int y;
    public karmasiksayi()
    {
        x = 0;
        y = 0;
    }
    public karmasiksayi(int m, int n)
    {
        x = m;
        y = n;
    }
    public string yaz()
    {
        return x + "i" + " + " + y;
    }

    Operatörlerimiz
}
```

Kişisel Operatör Tanımlama

```
//Tekil Operatör
public static karmasiksayi operator -(karmasiksayi c)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = -Math.Abs(c.x);
    temp.y = -Math.Abs(c.y);
    return temp;
}
```

- Karmaşık sayının tüm elemanlarını eksi yapma. Kendimize özel matematik dünyasına uymasına gerek yok. *-1 yapıp matematik dünyasına özel yapabilirdik. Kurallar:
  - Sınıf içinde operatör işlemi static olmak zorundadır.
  - Return edeceği değer olmalıdır.
  - Operator kelimesi olmalıdır.
  - -,+ vs. operatör türü olmalıdır.
  - Operatör işlemi yapılacak bir veya birden fazla sınıf olmalıdır.

Kişisel Operatör Tanımlama

```
//İkili Operatör
public static karmasiksayi operator -(karmasiksayi c1, karmasiksayi c2)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = c1.x - c2.x;
    temp.y = c1.y - c2.y;
    return temp;
}
```

- İki değişken arasına - işlemi yapıldığında operatörün davranması.
- Yeni sınıf oluşturmaya ve gelen değerleri yeni sınıfa atıp, yeni sınıfı return etmeye dikkat!!!

Kişisel Operatör Tanımlama

```
public class karmasiksayi
{
    private int x;
    private int y;
    public karmasiksayi()
    {
        x = 0;
        y = 0;
    }
    public karmasiksayi(int m, int n)
    {
        x = m;
        y = n;
    }
    public string yaz()
    {
        return x + "i" + " " + " + y;
    }

    Operatörlerimiz
}
```

```
#region Operatörlerimiz
//Tekil Operatör
public static karmasiksayi operator -(karmasiksayi c)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = -Math.Abs(c.x);
    temp.y = -Math.Abs(c.y);
    return temp;
}
public static karmasiksayi operator +(karmasiksayi c)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = Math.Abs(c.x);
    temp.y = Math.Abs(c.y);
    return temp;
}
//İkili Operatör
public static karmasiksayi operator -(karmasiksayi c1, karmasiksayi c2)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = c1.x - c2.x;
    temp.y = c1.y - c2.y;
    return temp;
}
public static karmasiksayi operator +(karmasiksayi c1, karmasiksayi c2)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = c1.x + c2.x;
    temp.y = c1.y + c2.y;
    return temp;
}
#endregion
```

Kişisel Operatör Tanımlama

```
#region Operatörlerimiz
//Tekil Operatör
public static karmasiksayi operator -(karmasiksayi c)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = -Math.Abs(c.x);
    temp.y = -Math.Abs(c.y);
    return temp;
}
public static karmasiksayi operator +(karmasiksayi c)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = Math.Abs(c.x);
    temp.y = Math.Abs(c.y);
    return temp;
}
//İkili Operatör
public static karmasiksayi operator -(karmasiksayi c1, karmasiksayi c2)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = c1.x - c2.x;
    temp.y = c1.y - c2.y;
    return temp;
}
public static karmasiksayi operator +(karmasiksayi c1, karmasiksayi c2)
{
    karmasiksayi temp = new karmasiksayi();
    temp.x = c1.x + c2.x;
    temp.y = c1.y + c2.y;
    return temp;
}
#endregion
```

```
class Program
{
    static void Main(string[] args)
    {
        karmasiksayi _ks1 = new karmasiksayi(-4, 6);
        karmasiksayi _ks2 = new karmasiksayi(2, -1);
        Console.WriteLine(_ks1.yaz());
        Console.WriteLine(_ks2.yaz());

        karmasiksayi _ks3 = -_ks1;
        Console.WriteLine(_ks3.yaz());

        karmasiksayi _ks4 = +_ks2;
        Console.WriteLine(_ks4.yaz());

        karmasiksayi _ks5 = _ks1 + _ks2;
        karmasiksayi _ks6 = _ks3 - _ks4;
        Console.WriteLine(_ks5.yaz());
        Console.WriteLine(_ks6.yaz());
    }
}
```

Overload Yapılabilen ve Yapılamayan Operatörler

- Tekil işlemler için olan operatörler ile overload yapılabilir:
  - +, -, !, ~, ++, --
- Aşağıdaki işlem opratörleri kullanılabilir:
  - +, -, *, /, %
- Karşılaştırma opratörleri ile overload edilebilir:
  - ==, !=, <, >, <=, >=
- Mantıksal operatörler overload edilemez.
  - &&, ||
- Atama operatörleri overload edilemez.
  - +=, -=, *=, /=, %=
- Bu opratörler overload edilemez.
  - =, ., ?:, ->, new, is, sizeof, typeof

Inheritance (Kalıtım, Miras alma)

- Inheritance (miras alma, kalıtım), bir nesnenin özelliklerinin farklı nesneler tarafından da kullanılabilmesine olanak sağlayan NYP özelliğidir.
- Yazılan bir sınıf bir başka sınıf tarafından miras alınabilir. Bu işlem yapıldığı zaman temel alınan sınıfın tüm özellikleri yeni sınıfa aktarılır.
- Örneğin:
  - İnsan – Canlı ilişkisinde, insanın Canlı sınıfını miras aldığı söylenebilir. Bu sayede insan sınıfını yazarken canlı özelliklerini tekrar yazmamıza gerek kalmaz. Örneğin nefes al fonksiyonu canlıya yazılır insana tekrar yazılmasına gerek yoktur.
  - Elinizde bir taşıt sınıfı varsa; otomobil, kamyon, motosiklet gibi alt sınıfları üretmek çok daha az çaba gerektirir. Taşıt sınıfına tekerlik diye bir özelliği olsun. Otomobil 4, kamyon 6 ve motorsiklet 2 tekerlekten oluşur dersek, her bir sınıf için tekerlek tanımlamak yerine taşıta tanımlandığımız tekerlek özelliği sayesinde üç sınıfı daha yönetebiliriz. Tekerlek özelliği taşıt sınıfından diğer sınıflara miras kalmış olur.

Kalıtım - Türetme

- Diyelim ki elinizde A diye bir sınıf var. Ancak B diye bir sınıf daha oluşturmanız gerekiyor. Ancak bu B sınıfının içinde A sınıfındaki özellik ve metotların da bulunması gerekiyor. İşte bu durumda B sınıfını A sınıfından türetmeniz gerekir. Türetme kalıtım yoluyla olduğu için A sınıfının bütün üye elemanları B sınıfına adeta kopyalanır. Daha sonra B sınıfının kendine özel üye elemanlarını yazabiliriz. Örneğin:

```
class A
{
    ...
}
class B:A
{
    ...
}
```

Kalıtım - Türetme

- .Net kütüphanesindeki birçok sınıf birbirlerinden türetilmiştir.
- Örneğin temel veri türleri dediğimiz byte, int, uint, short, float ve bezerlerinin tamamı object sınıfından türetilmiştir.
- Bu sayede normalde object sınıfında bulunan ToString() metodunu bu yapı türünden nesnelerde de kullanabilirsiniz.

Kalıtım - Erişim Belirteçleri

- public olarak tanımlanan öğe, kod bloğunun içinde ve dışında tamamen erişilebilirdir. Yani, hiçbir kısıtlama yoktur. Veri kontrol işlemi gerektiğinde veya güvenlik açısından önemli olan üyeler kesinlikle public tanımlanmamalıdır.
- private olarak tanımlanan öğe, sadece tanımlandığı class'ın içerisinde erişilebilirdir. En katı erişim belirleyicidir. Sadece kendi sınıfından o üyeye ulaşabiliriz. Sınıf dışından gereksiz müdahalelerin önüne geçmiş oluruz. Ayrıca bie fonksiyon veya property üzerinden bu değişkenleri yönetebiliriz. Bu yüzden kontrol ve güvenlik açısından bize destek sağlar.
- Protected ve internal sadece kalıtımda kullanılmaktadır.

Kalıtım - Erişim Belirteçleri

- protected : protected olarak tanımlanan öğe, sadece tanımlandığı class'ın içinde ve o class'tan türetilmiş diğer class'ların içinde erişilebilirdir.
- internal : internal olarak tanımlanan öğe, bulunduğu assembly'nin (DLL veya EXE dosyası) içinde erişilebilirdir. DLL veya EXE dosyasının içerisinde erişim için kısıtlama yoktur, ama dışarıdan erişilemez.
- protected internal : protected internal erişim belirleyicisi, protected ve internal erişim belirleyicilerinin VEYA (OR) işlemiyle birleştirilmiş halidir. protected internal olarak tanımlanmış öğe, tanımlandığı class'ın içinde ve o class'tan türetilmiş diğer class'ların içinde erişilebilir. Ayrıca, aynı assembly içinde olmasalar dahi, tanımlandığı class'tan türetilmiş diğer class'ların içinde de erişilebilirdir.

Kalıtım - Erişim Belirteçleri

- Eğer erişim belirteci kullanılmazsa
  - class public, ya da internal olabilir, varsayılan olarak internal erişim belirleyicisine sahiptirler.
  - Varsayılan olarak class içerisinde tanımlı öğeler private erişim belirleyicisine sahiptirler.

Base - this

- base : türetmede kullanılan sınıfı temsil eder.
- this : o anda içinde bulunduğunuz sınıfı temsil eder.

Örneğin:

  - A sınıfı B sınıfından türetilmişse B sınıfında kullanılan bir this ifadesi iki sınıfı da kapsar.
  - B sınıfında base kullanılırsa sadece A sınıfını kapsar.

Kalıtım Örneği

```
public class X
{
    public int m;
    private int k; //alt sınıftan ulaşılamaz
    protected int z = 10; //kendinden ve alt sınıftan ulaşılır.

    public int topla1()
    {
        return this.m + z;
        //n ve z1 ulaşılamaz
    }
}
public class Y : X
{
    public int n;
    protected int z1;

    public int topla2()
    {
        return base.m + this.n + z;
    }
}
class Program
{
    static void Main(string[] args)
    {
        Y _y = new Y();
        _y.m = 10;
        _y.n = 5;
        Console.WriteLine(_y.topla2());
        Console.ReadLine();
    }
}
```

Y sınıfından çağrılabilir.
Main bölümünden çağrılamaz.

Sadece X sınıfını kapsar.

Y sınıfı X sınıfın özelliklerini kullanıyor. Base sınıf X'tir.

Sadece X sınıfına ulaşır.

Hem X hem de Y sınıflarına ulaşır.

Yapıcılar ve Kalıtım

- C#'ta yapıcı metotların türetimiyle ilgili şu kurallar geçerlidir:
  - C#'ta yapıcı metotlar fiziksel olarak türetilmez.
  - Yavru sınıf türünden bir nesne yaratıldığında önce ana sınıfın parametre almayan yapıcı metodu, ardından yavru sınıftaki imzaya uyan yapıcı metot çalıştırılır.
  - Yavru sınıf türünden nesne yaratımında daima yavru sınıfın imzaya uyan bir yapıcı metodu olması gerekir.
  - Yavru sınıf türünden nesne yaratımlarında, ana sınıfın parametre almayan yapıcı metodu yavru sınıfın üye elemanlarıyla işlem yapar.
  - Yavru sınıf türünden nesne yaratımında, yavru sınıftaki ilgili (imzası uygun) yapıcı metoda base takısı eklenmişse ana sınıfın parametre almayan yapıcı metodu çalıştırılmaz.

Yapıcılar ve Kalıtım

```
using System;
class ana
{
   public ana()
   {
      Console.WriteLine("ana sınıfının parametre almayan yapıcı metodu");
   }
}
class yavru:ana
{
   public yavru(int a)
   {
      Console.WriteLine("yavru sınıfının parametre alan yapıcı metodu. alınan parametre: "+a);
   }
}
class esas
{
   static void Main()
   {
      yavru y=new yavru(5);
   }
}
```

- Madde 2:
  - Bu programda ekrana alt alta ana sınıfının parametre almayan yapıcı metodu ve yavru sınıfının parametre alan yapıcı metodu. alınan parametre: 5 yazacaktır.

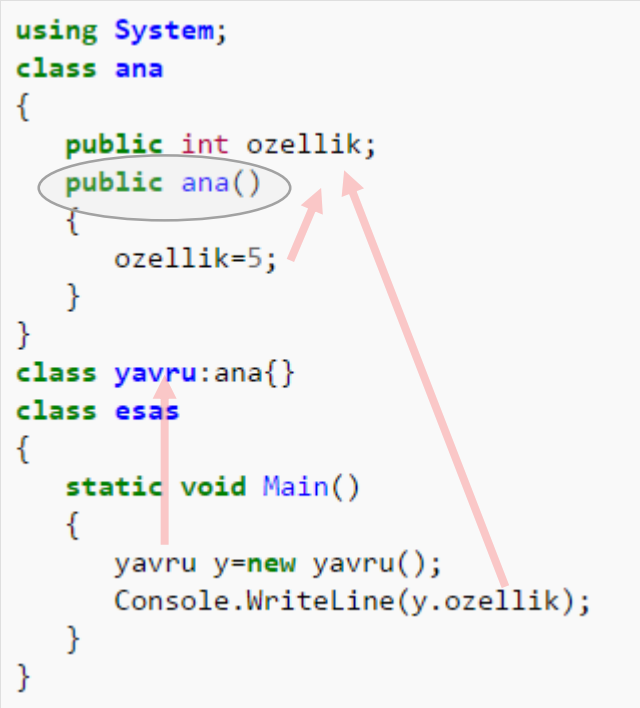
Not: Constructor içinde yazan Console.WriteLine sadece gösterim amaçlıdır.
Bir sınıf tasarımında sınıf içinde Console.WriteLine önermem… Çünkü ileri de sadece Console'a değil bir çok ortama kod yazacağız. Aynı kodu farklı ortamlardan kullanmak için Sınıfımızı iyi tasarlamalıyız.

Yapıcılar ve Kalıtım

```
using System;
class ana
{
   public ana()
   {
      Console.WriteLine("ana sınıfının parametre almayan yapıcı metodu");
   }
}
class yavru:ana{}
class esas
{
   static void Main()
   {
      yavru y=new yavru();
   }
}
```

- Bu program hata vermez. Çünkü sınıflar konusunda öğrendiğimiz üzere bir sınıfta hiç yapıcı metot olmadığı durumlarda varsayılan yapıcı metot oluşturulur. Varsayılan yapıcı metot parametre almaz ve hiçbir şey yapmaz. Bu örnekte y nesnesini yaratırken parametre verseydik yukarıda bahsettiğimiz üçüncü kural ihlal edilmiş olacaktı ve dolayısıyla programımız hata verecekti.

Yapıcılar ve Kalıtım



```
using System;
class ana
{
   public int ozellik;
   public ana()
   {
      ozellik=5;
   }
}
class yavru:ana{}
class esas
{
   static void Main()
   {
      yavru y=new yavru();
      Console.WriteLine(y.ozellik);
   }
}
```

- Bu örnekte ekrana 5 yazılacaktır. Aslına bakarsanız dördüncü kuralın tersinin olması imkansız. Çünkü zaten ortada ana tipinden bir nesne yok, o yüzden ana sınıfının parametre almayan yapıcı metodunun kendi sınıfının üye elemanlarıyla çalışması bu örnekte imkansız. Ancak yine de fiziksel olarak bir sınıfta olmayan bir yapıcı metodun o sınıfın üye elemanlarıyla çalışması ilginç.

Yapıcılar ve Kalıtım

```
using System;
class ana
{
   public ana(int a){}
}
class yavru:ana{}
class esas
{
   static void Main()
   {
      yavru y=new yavru();
   }
}
```

- Bu program hata verir. Çünkü yukarıda saydığımız ikinci kuralı ihlal etmektedir. Çünkü ana sınıfta parametre almayan bir yapıcı metot yoktur. Ana sınıfta bir yapıcı metot tanımlandığı için varsayılan yapıcı metot oluşturulmamıştır.

Yapıcılar ve Kalıtım

```
using System;
class A
{
   public int Ozellik1;
   public int Ozellik2;
   public A()
   {
      Console.WriteLine("Deneme"),
   }
   public A(int ozellik1,int ozellik2)
   {
      Ozellik1=ozellik1;
      Ozellik2=ozellik2;
   }
}
class B:A
{
   public int Ozellik3;
   public int Ozellik4;
   public B(int ozellik3,int ozellik4,int ozellik1,int ozellik2):base(ozellik1,ozellik2)
   {
      Ozellik3=ozellik3;
      Ozellik4=ozellik4;
   }
}
class esas
{
   static void Main()
   {
      B b=new B(3,4,1,2);
      Console.WriteLine(b.Ozellik1+" "+b.Ozellik2+" "+b.Ozellik3+" "+b.Ozellik4);
   }
}
```

- Bu program ekrana 1 2 3 4 yazar. Bu örnekte base anahtar sözcüğü ana sınıftaki yapıcı metodu temsil etmektedir. Örneğimizde yavru sınıfın yapıcı metodu 4 parametre almakta, bu aldığı parametrelerin ikisini kendi bloğunda kullanmakta kalan iki parametreyi de ana sınıfın imzası uygun yapıcı metoduna göndermektedir. Ana sınıfın imzası uygun yapıcı metodu çalıştığında yavru sınıfın üye elemanlarıyla işlem yapacaktır. Asıl konumuza gelecek olursak bu örnekte yavru sınıfın bir yapıcı metoduna eklenen base takısı ile ana sınıfın bir yapıcı metodunu çalıştırdığımız için yavru sınıftaki base takısı eklenmiş ilgili yapıcı metodu çalıştıracak şekilde yavru sınıf türünden bir nesne yaratıldığında ana sınıfın parametre almayan yapıcı metodu çalıştırılmayacaktır.

Yapıcılar ve Kalıtım

```
using System;
class A
{
   public A()
   {
      Console.WriteLine("A sınıfı");
   }
}
class B:A
{
   public B():base()
   {
      Console.WriteLine("B sınıfı");
   }
}
class Ana
{
   static void Main()
   {
      B b=new B();
   }
}
```

Olmasa da olur.
Default (Geçerli) Durumdur.

- NOT1: base anahtar sözcüğü bu şekilde yalnızca yapıcı metotlarla kullanılabilir. Yani base anahtar sözcüğünü yalnızca yavru sınıftaki yapıcı metoda ekleyebiliriz ve base anahtar sözcüğünün ana sınıfta var olan bir yapıcı metodu belirtmesi gerekir.
- NOT2: Yavru sınıfın yapıcı metoduna base takısı eklenip ana sınıfın bir yapıcı metodu çalıştırıldığı durumlarda önce ana sınıfın ilgili (imzaya uyan) yapıcı metodu, ardından yavru sınıfın ilgili (imzaya uyan) yapıcı metodu çalıştırılır.

Yapıcılar ve Kalıtım

- Bu programda alt alta A sınıfı ve B sınıfı yazılacaktır. Şimdi isterseniz bir de statik yapıcı metotların kalıtımdaki rolüne bakalım.

```
B sınıfının statik yapıcı metodu
A sınıfının statik yapıcı metodu
A sınıfının statik olmayan yapıcı metodu
B sınıfının statik olmayan yapıcı metodu
-------
A sınıfının statik olmayan yapıcı metodu
B sınıfının statik olmayan yapıcı metodu
```

```
using System;
class A
{
   static A()
   {
      Console.WriteLine("A sınıfının statik yapıcı metodu");
   }
   public A()
   {
      Console.WriteLine("A sınıfının statik olmayan yapıcı metodu");
   }
}
class B:A
{
   static B()
   {
      Console.WriteLine("B sınıfının statik yapıcı metodu");
   }
   public B()
   {
      Console.WriteLine("B sınıfının statik olmayan yapıcı metodu");
   }
}
class Ana
{
   static void Main()
   {
      B b1=new B();
      Console.WriteLine("-------");
      B b2=new B();
   }
}
```

Static'ler başlangıçta çalışır. Daha sonra çalışmaz.

Harici yapıcılar her nesne oluşumda çalışır.

Çoklu Türetmeler

- Sınıflar tıpkı nine, anne, çocuk yapısında olduğu gibi ard arda türetilebilir. Yani örneğin B sınıfı A sınıfında türetilip C sınıfı da B sınıfından türetilebilir. Bu durumda C sınıfı türünden bir nesne yarattığımızda eğer C sınıfının ilgili yapıcı metoduna base takısını eklememişsek önce A, sonra B, sonra da C sınıfının yapıcı metotları çalıştırılır. Yani gidişat anadan yavruya doğrudur. Ayrıca tahmin edebileceğiniz gibi C sınıfı hem A'nın hem de B'nin bütün üye elemanlarına sahip olur.

```
using System;
class A
{
    public A()
    {
        Console.WriteLine("A sınıfı");
    }
}
class B:A
{
    public B()
    {
        Console.WriteLine("B sınıfı");
    }
}
class C:B
{
    public C()
    {
        Console.WriteLine("C sınıfı");
    }
    static void Main()
    {
        C nesne=new C();
    }
}
```

Çoklu Türetmeler

- Bu program ekrana alt alta A sınıfı, B sınıfı ve C sınıfı yazacaktır. Bu örnekte base anahtar sözcüğünün kullanımı ise şöyledir:

```
using System;
class A
{
   public int OzellikA;
   public A(int a)
   {
      OzellikA=a;
   }
}
class B:A
{
   public int OzellikB;
   public B(int b,int a):base(a)
   {
      OzellikB=b;
   }
}
class C:B
{
   public int OzellikC;
   public C(int c,int b,int a):base(b,a)
   {
      OzellikC=c;
   }
   static void Main()
   {
      C nesne=new C(12,56,23);
      Console.WriteLine(nesne.OzellikA+" "+nesne.OzellikB+" "+nesne.OzellikC);
   }
}
```

Çoklu Türetmeler

- Gördüğünüz gibi base anahtar sözcüğü kendisinin bir üstündeki sınıfın yapıcı metoduna parametre gönderiyor ancak tabii ki işlemler alt sınıftaki üye elemanlar için yapılıyor. Bu durumda -hangi sınıf türünden nesne yaratırsak yaratalım- hiçbir sınıfın parametre almayan yapıcı metodu çalıştırılmayacaktır (tabii ki nesne yaratırken gerekli parametreleri verdiğimiz müddetçe). Bu örneği şöyle değiştirirsek program hata verir:

```
using System;
class A
{
   public int OzellikA;
   public A(int a)
   {
      OzellikA=a;
   }
}
class B:A
{
   public int OzellikB;
   public B(int b)
   {
      OzellikB=b;
   }
}
class C:B
{
   public int OzellikC;
   public C(int c,int b):base(b)
   {
      OzellikC=c;
   }
   static void Main()
   {
      C nesne=new C(12,56);
      Console.WriteLine(nesne.OzellikA+" "+nesne.OzellikB+" "+nesne.OzellikC);
   }
}
```

Base şart çünkü A'da parametreli bir constructor var.

İsim saklama

- Muhtemelen şunu merak etmişsinizdir: yavru sınıfta ana sınıftakiyle aynı isimli bir üye eleman varsa ne olacak? İşte bu durumda ana sınıftaki üye eleman gizlenir ve normal yöntemlerle erişilemez. Buna isim saklama denir. Örnek:

```
using System;
class A
{
   public int a=10;
}
class B:A
{
   public int a=20;
    static void Main()
    {
        B nesne=new B();
        Console.WriteLine(nesne.a);
    }
}
```

İsim saklama

- Bu programda ekrana 20 yazılır yani değeri 20 olan a özelliği, değeri 10 olan a özelliğini gizlemiştir. Ancak derleyici bu programda bizim isim gizlemeyi bilinçsiz olarak yaptığımızı düşünür ve uyarı verir (hata vermez). Böyle bir durumda isim gizlemeyi açıkça belirtmeliyiz. Böylelikle hem programımızdaki muhtemel gözden kaçmaları hem de derleyicinin uyarı vermesini önleriz. İsim gizlemeyi açıkça belirtmek için new anahtar sözcüğünü kullanırız. Yukarıdaki programı şöyle yazarsak derleyici uyarı vermez:

```
using System;
class A
{
   public int a=10;
}
class B:A
{
   public new int a=20;
    static void Main()
    {
        B nesne=new B();
        Console.WriteLine(nesne.a);
    }
}
```

İsim saklama

- Peki gizlediğimiz üye elemana erişebilir miyiz? Cevabımız evet. Bunun için base anahtar sözcüğünü kullanırız. Örnek:

```
using System;
class A
{
    public int a=10;
}
class B:A
{
    public new int a=20;
    int Metot()
    {
        return base.a;
    }
    static void Main()
    {
        B nesne=new B();
        Console.WriteLine(nesne.Metot());
    }
}
```

Burada ekrana 10 yazılır. base anahtar sözcüğü static metotların içinde kullanılamaz.

Destructor (Yıkıcı) ve Kalıtım

- Bir C sınıfı yarattığımızda Yapıcı A,B,C şeklinde devam ederken Yıkıcı işlemi C, B, A şeklinde devam eder.
- GC.Collect'i yorum satırını silip deneyebilirsiniz. (Ayrıntısı geçen haftaki konularda)

```
Creating A
Creating B
Creating C
Object Created
Press enter to Destroy it
Destroying C
Destroying B
Destroying A
```

```
using System;
class A
{
 public A()
 {
  Console.WriteLine("Creating A");
 }
 ~A()
 {
  Console.WriteLine("Destroying A");
 }
}

class B:A
{
 public B()
 {
  Console.WriteLine("Creating B");
 }
 ~B()
 {
  Console.WriteLine("Destroying B");
 }

}
class C:B
{
 public C()
 {
  Console.WriteLine("Creating C");
 }

 ~C()
 {
  Console.WriteLine("Destroying C");
 }
}
class App
{
 public static void Main()
 {
  C c=new C();
  Console.WriteLine("Object Created ");
  Console.WriteLine("Press enter to Destroy it");
  Console.ReadLine();
  c=null;
  //GC.Collect();
  Console.Read();
 }

}
```

Kaynaklar

- http://www.codeproject.com/Articles/602141/Polymorphism-in-NET
- https://gelecegiyazanlar.turkcell.com.tr/konu/windows-phone/egitim/windows-phone-101/miras-alma-kalitim-inheritance
- https://tr.wikibooks.org/wiki/C_Sharp_Programlama_Dili/Kal%C4%B1t%C4%B1m
- http://www.enginpolat.com/csharp-erisim-belirleyiciler-access-modifiers/
- https://tr.wikibooks.org/wiki/C_Sharp_Programlama_Dili/Kal%C4%B1t%C4%B1m
- http://www.codeproject.com/Articles/7792/Destructors-in-C